TP 6 (2020) Systèmes concurrents et distribués

Exercice 1 :

Avec 2 consommateurs et 1 producteur observez un interblocage. Proposez une exécution qui résulte en un interblocage pour l'implémentation de la file d'attente ci-dessous si QSIZE=1 et 2 threads exécutent ontinuellement dec() et un thread enc() .

```
public class queue {
	private int head = 0, tail = 0;
	final int QSIZE=1;
	Item[QSIZE] items;

	public synchronized void enq(Item x) {
		while (this.tail – this.head == QSIZE)
			this.wait();
		this.items[this.tail++%QSIZE] = x;
		this.notify();
	}

	public synchronized Item deq(Item x) {
		while (this.tail – this.head == 0)
			this.wait();
		Item inter =  this.items[this.head%QSIZE];
		this.head+=1;
		this.notify();
		return inter;

	}
}
```

Exercice 2 :

Montrez que l'implémentation des sémaphore binaires vue au cours n'assure pas la vivacité.

Ecrire une implémentation des fonctions P() et V() pour une sémaphore binaire qui assure qu'un thread qui appelle P() entrera en section critique (pas de famine).

Idée : utilisez deux files d'attente. Les threads dans la seconde file d'attente sont servis d'abord. Lorsqu'il n'y a plus de threads dans la seconde file d'attente, tous les threads dans la première passent dans la seconde.